

CCF 全国信息学奥林匹克联赛（NOIP2019）模拟题

提高组

(请选手务必仔细阅读本页内容)

一. 题目概况

中文题目名称	小 R 与糖果	小 R 与旅行	小 R 与 b 图
英文题目与子目录名	candy	tour	b
可执行文件名	candy	tour	b
输入文件名	candy.in	tour.in	b.in
输出文件名	candy.out	tour.out	b.out
每个测试点时限	1s	1s	1s
测试点数目	10	10	10
每个测试点分值	10	10	10
附加样例文件	有	有	有
结果比较方式	全文比较（过滤行末空格及文末回车）		
题目类型	传统	传统	传统
运行内存上限	256MB	256MB	256MB

二. 提交源程序文件名

对于 C++ 语言	candy.cpp	tour.cpp	b.cpp
对于 C 语言	candy.c	tour.c	b.c
对于 pascal 语言	candy.pas	tour.pas	b.pas

三. 编译命令（不包含任何优化开关）

对于 C++ 语言	g++ -o candy candy.cpp -lm	g++ -o tour tour.cpp -lm	g++ -o b b.cpp -lm
对于 C 语言	gcc -o candy candy.c -lm	gcc -o tour tour.c -lm	gcc -o b b.c -lm
对于 pascal 语言	fpc candy.pas	fpc tour.pas	fpc b.pas

注意事项:

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. c/c++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值是 0。
3. 全国统一评测时采用的机器配置为：CPU AMC Athlon(tm) II*2 240 processor, 2.8GHz，内存 4G，上述时限以此配置为准。
4. 只提供 Linux 格式附加样例文件。
5. 提交的程序代码文件的放置位置请参照各省的具体要求。
6. 特别提醒：评测在 Windows 下使用 Lemon 进行，各语言的编译器版本以其为准。

1. 小 R 与糖果

(candy.cpp/c/pas)

【问题描述】

小 R 喜欢吃糖果。

小 R 有 n 颗糖果，每颗糖果可以提供一个欢乐值。小 R 会一种神奇的魔法，每次施展魔法可以使他多得到一颗糖果，这颗糖果的欢乐值为 $x+y$ ，其中 x 和 y 分别是小 R 已有的两颗糖果的欢乐值。小 R 在吃掉这些糖果之前想先施展 k 次魔法（必须施展 k 次）。现在小 R 想知道他最多能获得的欢乐值的和是多少。

【输入格式】

第一行有两个整数 n, k 表示初始糖果数量和施展魔法的次数，第二行包含 n 个整数表示初始时小 R 的糖果。

【输出格式】

输出一个整数，表示欢乐值之和的最大值。答案对 10000007 取模。

【输入输出样例】

candy.in	candy.out
2 2	33
3 6	

见选手目录下的 candy / candy.in 和 candy / candy.out

【数据规模及约定】

对于 60% 的数据，有 $n \leq 10^5$ ， $k \leq 10^5$ ， $|a_i| \leq 10^5$ 。

对于 100% 的数据，有 $n \leq 10^5$ ， $k \leq 10^9$ ， $|a_i| \leq 10^5$ 。

2. 小 R 与旅行

(tour.cpp/c/pas)

【问题描述】

小 R 赢得了一张 L 国家环游机票。旅行在这家航空公司开放的最西边的城市开始，然后一直自西向东旅行，直到你到达最东边的城市，再由东向西返回，直到你回到开始的城市。除了旅行开始的城市之外，每个城市只能访问一次，因为开始的城市必定要被访问两次（在旅行的开始和结束）。当然不允许使用其他公司的航线或者用其他的交通工具。

给出这个航空公司开放的城市的列表，和两两城市之间的直达航线列表。找出能够访问尽可能多的城市的路线，这条路线必须满足上述条件，也就是从列表中的第一个城市开始旅行，访问到列表中最后一个城市之后再返回第一个城市。

【输入格式】

第 1 行：航空公司开放的城市数 N 和将要列出的直达航线的数量 M 。
第 2.. $N+1$ 行：每行包括一个航空公司开放的城市名称。城市名称按照自西向东排列。不会出现两个城市在同一条经线上的情况。每个城市的名称都是一个字符串，最多 15 字节，由拉丁字母表上的字母组成；城市名称中没有空格。
第 $N+2$.. $N+2+M-1$ 行：每行包括两个城市名称（由上面列表中的城市名称组成），用一个空格分开。这样就表示两个城市之间的直达双程航线。

【输出格式】

第 1 行：按照最佳路线访问的不同城市的数量。如果无法找到路线，输出 1。

【输入输出样例】

tour. in	tour. out
8 9 Vancouver Yellowknife Edmonton Calgary Winnipeg Toronto Montreal Halifax Vancouver Edmonton Vancouver Calgary Calgary Winnipeg Winnipeg Toronto Toronto Halifax Montreal Halifax Edmonton Montreal Edmonton Yellowknife	7

Edmonton Calgary	
------------------	--

见选手目录下的 tour / tour.in 和 tour / tour.out

【数据规模及约定】

对于 30%的数据， $n \leq 10$ ， $m \leq 20$

对于 60%的数据， $n \leq 50$ ， $m \leq 1200$

对于 100%的数据， $n \leq 500$ ， $m \leq 1500$

3. 小 R 与 b 图

b. cpp/c/pas

【问题描述】

小 R 喜欢探险。

一天他来到 b 国，闯过这个国家需要携带一些精灵。小 R 把 b 国抽象成一张有 n 个点 m 条边的无向连通图--b 图，无重边和自环，节点编号为 $1\sim n$ 。每个节点有一个精灵需求数， u 节点的精灵需求数记为 $w[u]$ 。通过这些边需要支付一定数量的精灵，费用规定如下：1) 普通道路（类别为 0），需要支付 1 只精灵；2) 特殊道路（类别为 1），需要支付自己当前携带的精灵数的 $1/20$ （不足 20 的按 20 只计算，例如，若他身上有 10 只精灵，需支付 1 只；有 69 只精灵，需支付 4 只）。

小 R 有点闲，觉得只找出需要精灵数最少的路径太简单，就定义了一个整数 S ，表示每对节点之间的最优路径长度之和。我们用 $d(u,v)$ 表示 u, v 之间的最优路径长度， $d(u,v)$ 被定义为从 u 沿路径到达 v ，且身上至少还有 $w[v]$ 只精灵的前提下，初始携带的最少精灵数。

小 R 觉得只求 S 还是太简单了，于是想求删除一条边之后新的 S 值 S' 的最大值，当然需要保证删除后的图仍然联通。但小 R 还有点咸，所以不会求 S' 的他只能来求助你了。

【输入格式】

第一行包含两个正整数 n 和 m ；

第二行包括 n 个整数 $w_1\cdots w_n$ ；

接下来 m 行，每行包括三个整数 u,v,op ，代表一条连接 u,v 的无向边，类别为 op 。

【输出格式】

第一行包括两个整数，为 S 和最大的 S' 。

【输入输出样例】

b. in	b. out
5 6 15 14 10 14 12 1 2 0 2 3 0 3 4 1 2 5 1 3 5 1 3 1 1	353 357

见选手目录下的 b / b.in 和 b / b.out

【数据规模及约定】

对于 30%的数据, $n \leq 5$, $m \leq 8$, $op=0$

对于 60%的数据, $n \leq 70$, $m \leq 140$, $op=0$

对于 100%的数据, $n \leq 70$, $m \leq 140$, $1 \leq w_i \leq 100$